Report for Project 1 in INF264: Decision Trees

Eirik Aarrestad

Table of Contents

1 Introduction **1**

2 Data and methods **2**
2.1 The dataset 2
2.2 Implementation of the decision tree 2
2.3 Data split 3
2.4 Model selection 3
2.5 Comparison with scikit-learn 4
2.6 Permutation importance 4

3 Results / findings **4**
3.1 Model selection 4
3.2 Evaluation on the test set 5
3.3 Permutation importance 6

4 Conclusion **8**

1 Introduction

In this project we implement a decision tree classifier from scratch and use it to predict customer churn in a dataset from a telecom company. The dataset consists of 7032 customers and 19 features, including contract type, how long the customer has been with the company, monthly charges and which additional services the customer has. The target variable `Churn` indicates whether the customer left the company (1) or stayed (0). Three of the features are numerical, while the rest are categorical and already encoded as integers.

The project consists of three parts. In part 1 we implement the ID3 algorithm with entropy or Gini as the impurity measure, and with maximum depth as a hyperparameter. In part 2 we select hyperparameters using cross-validation and compare our own implementation with

`DecisionTreeClassifier` from scikit-learn on a test set. In part 3 we implement permutation importance and use it to investigate which features our best model depends on the most.

2 Data and methods

2.1 The dataset

We load `churn-data.csv` with `numpy.genfromtxt`, as suggested in the project description, which gives a matrix `X` with 7032 rows and 19 columns and a vector `y` with the class labels. According to the project description the dataset has already been preprocessed: categorical features are already encoded as integers and there are no missing values, so we do no further preprocessing. The classes are imbalanced where churn is the smaller class, which effects the choice of evaluation metric and the interpretation of the results.

2.2 Implementation of the decision tree

We implement ID3 recursively in the class `DecisionTree` in the file `decision_tree.py`. The constructor takes two arguments: `criterion` (`"entropy"` by default, or `"gini"`) and `max_depth` (`None` by default). The method `fit(X, y)` builds the tree, and `predict(X)` sends each observation down the tree until it reaches a leaf node, which gives the predicted class.

At each node the algorithm creates a leaf node in the following situations:

- all observations have the same class (in which case this class is used)
- all observations have identical values on all features
- the maximum depth has been reached
- no feature gives a valid split.

In the last three cases the prediction is the most common class in the node. The last stopping criterion is an addition to the two described in the project description, and is needed because a split can become invalid when many observations share the same value. Otherwise, we choose the feature that gives the largest information gain, split the data in two and call the algorithm recursively on each part.Observations with a value $\leq$ the threshold go left and the rest go right, so each split gives two branches. Splits where one of the parts becomes empty are skipped.

The impurity I of a node with class proportions p_k is computed either as entropy or as the Gini index:

$$H(y) = -\sum_k p_k \log_2 p_k \qquad\qquad G(y) = \sum_k p_k(1 - p_k)$$

The information gain for a split of n observations into a left part with n_L and a right part with n_R observations is the reduction in weighted impurity:

$$IG = I(y) - \frac{n_L}{n} I(y_L) - \frac{n_R}{n} I(y_R)$$

For the Gini index, "information gain" therefore means the reduction in Gini impurity. We have also written a method `print_tree` that prints the tree in a readable format, as an aid for debugging.

2.3 Data split

We first split the dataset into a training set (80 %) and a test set (20 %) using `train_test_split`, stratified on `y` and with `random_state=42`. We do not touch the test set again until the final evaluation. All model selection is done on the training set only, so that the test result gives an unbiased estimate of how well the model generalizes to new data. The stratification ensures that the proportion of churn is approximately equal in the two parts, which is important since the classes are imbalanced. Instead of a separate validation set we use cross-validation on the training set, as described below.

2.4 Model selection

We tune two hyperparameters: `max_depth` and `criterion`. Maximum depth is the most important control on model complexity. A deep tree can fit the training data almost perfectly and overfits easily, while a tree that is too shallow does not capture the patterns in the data. We test $\{3, 5, 7, 10, \text{None}\}$ to cover everything from very simple trees to a fully grown tree, which serves as a reference for what happens without any form of restriction. For `criterion` we test both `entropy` and `gini`, since these are the two impurity measures the implementation supports.

Each combination is evaluated with 5-fold stratified cross-validation on the training set (`StratifiedKFold` with `random_state=42`). The training set is split into five parts where the class distribution is preserved in each part. The model is trained on four parts and evaluated on the fifth, and this is repeated until every part has been the validation set once. The mean of the five results is used as the performance measure for the combination. We use cross-validation rather than a single fixed validation set because the result then does not depend on which observations happen to end up in the validation set, and because all of the training data is used for both training and validation.

As the performance measure we use accuracy, the proportion of correctly classified customers and its is easy to interpret. Since the classes are imbalanced, accuracy can give an overly optimistic picture of how well the model finds the customers who actually leave. We also

report the F1 score for the churn class on the test set. F1 is used only for reporting and not for the model selection itself.

The combination with the highest mean accuracy is selected. We then train a new model with this combination on the whole training set and evaluate it once on the test set. All random choices (data split, folds and the seed in permutation importance) are fixed with `random_state` or `seed` equal to 42, and the whole experiment is run from `run_experiments.ipynb`, so that all numbers in the report can be reproduced.

2.5 Comparison with scikit-learn

To get a fair comparison we carry out exactly the same model selection for `DecisionTreeClassifier` from scikit-learn: the same grid of hyperparameters, the same evaluation metric, and the same folds, since both implementations are run through the same function `run_cv_grid` with the same seed for the folds. The scikit-learn model gets `random_state=42`.

2.6 Permutation importance

We implement `permutation_importance(model, X, y, metric, n_repeats, seed)` in `decision_tree.py` following the algorithm in the scikit-learn documentation. First the reference score s of the trained model on the unmodified data is computed. Then, for each feature j and each repetition k, the values in column j are randomly shuffled and the model's score $s_{k,j}$ is computed on the corrupted data. The importance of feature j is the mean drop in score:

$$e_j = s - \frac{1}{K} \sum_{k=1}^{K} s_{k,j}$$

The function returns the mean, the standard deviation and the raw values for each feature and repetition. The randomness is controlled by a `RandomState` initialized with `seed`. We compute the importance for the best of our own models on the test set, with accuracy as the metric, $K = 30$ repetitions and `seed=42`.

3 Results / findings

3.1 Model selection

Table 1 shows the mean accuracy from the cross-validation for all ten combinations, for both our own implementation and scikit-learn.

Table 1: Mean accuracy in 5-fold cross-validation on the training set

Max depth	Own: entropy	Own: gini	Sklearn: entropy	Sklearn: gini
3	0.7833	0.7833	0.7835	0.7835
5	0.7929	**0.7943**	0.7899	**0.7922**
7	0.7915	0.7888	0.7849	0.7838
10	0.7591	0.7572	0.7668	0.7653
None	0.7337	0.7273	0.7348	0.7401

As shown in **Table 1** and **Figure 1**, accuracy increases from depth 3 to depth 5, and then drops markedly for depth 10 and for the fully grown tree. This is a clear sign of overfitting. Both implementations perform best with `criterion="gini"` and `max_depth=5`, with a mean accuracy of 0.7943 for our implementation and 0.7922 for scikit-learn.

The difference between entropy and Gini is small. At depth 5 the difference is 0.0014 for our implementation and 0.0023 for scikit-learn, and at depth 3 the results are identical for both criteria in both implementations, which suggests that the criteria choose the same splits in such shallow trees. The choice of criterion is not as important as the choice of depth.

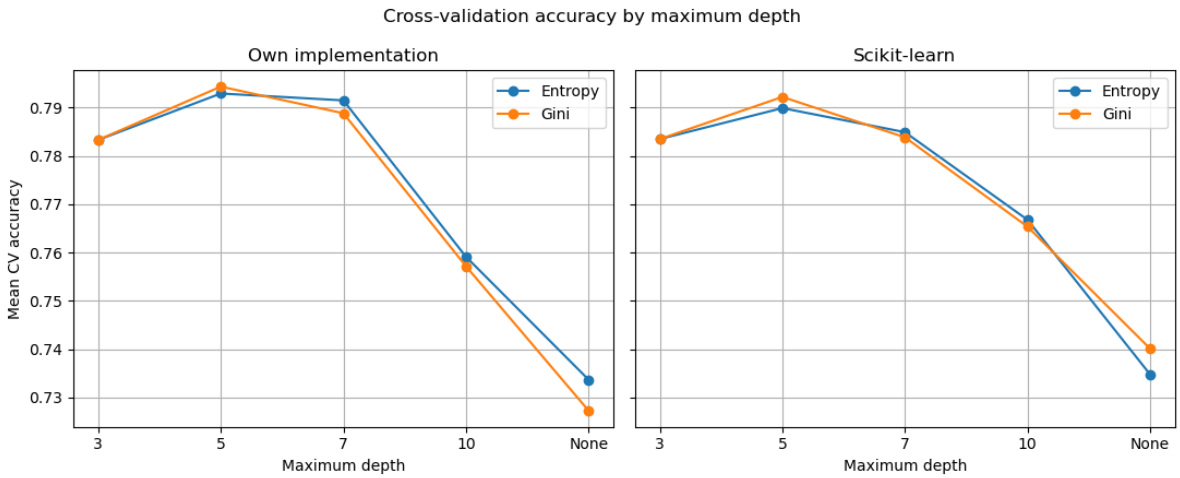


Figure 1: Mean cross-validation accuracy for different maximum depths

3.2 Evaluation on the test set

The selected models were trained on the whole training set and evaluated once on the test set of 1407 customers. The results are summarized in **Table 2**.

Table 2: Comparison of the selected models (both with gini and max depth 5)

Model	CV accuracy	Test accuracy	Test F1 (churn)
Own implementation	0.7943	0.7825	0.5248
scikit-learn	0.7922	0.7818	0.5492

The two implementations achieve almost identical accuracy on the test set: 0.7825 versus 0.7818. Test accuracy is about one percentage point lower than CV accuracy for both models. This is expected, since the hyperparameters were chosen to maximize the CV score, which is therefore slightly optimistic, while the test set is an independent sample.

The F1 score for the churn class is much lower, 0.5248 for our implementation and 0.5492 for scikit-learn. The models find the customers who actually leave a lot less then the overall accuracy of about 78 % would suggest, which is a consequence of the imbalanced classes. Scikit-learn scores slightly higher here (0.0244).

3.3 Permutation importance

Figure 2 shows the importance of each feature for the best of our own models, measured as the mean drop in accuracy on the test set when the feature is shuffled. The error bars show the standard deviation over the 30 repetitions.

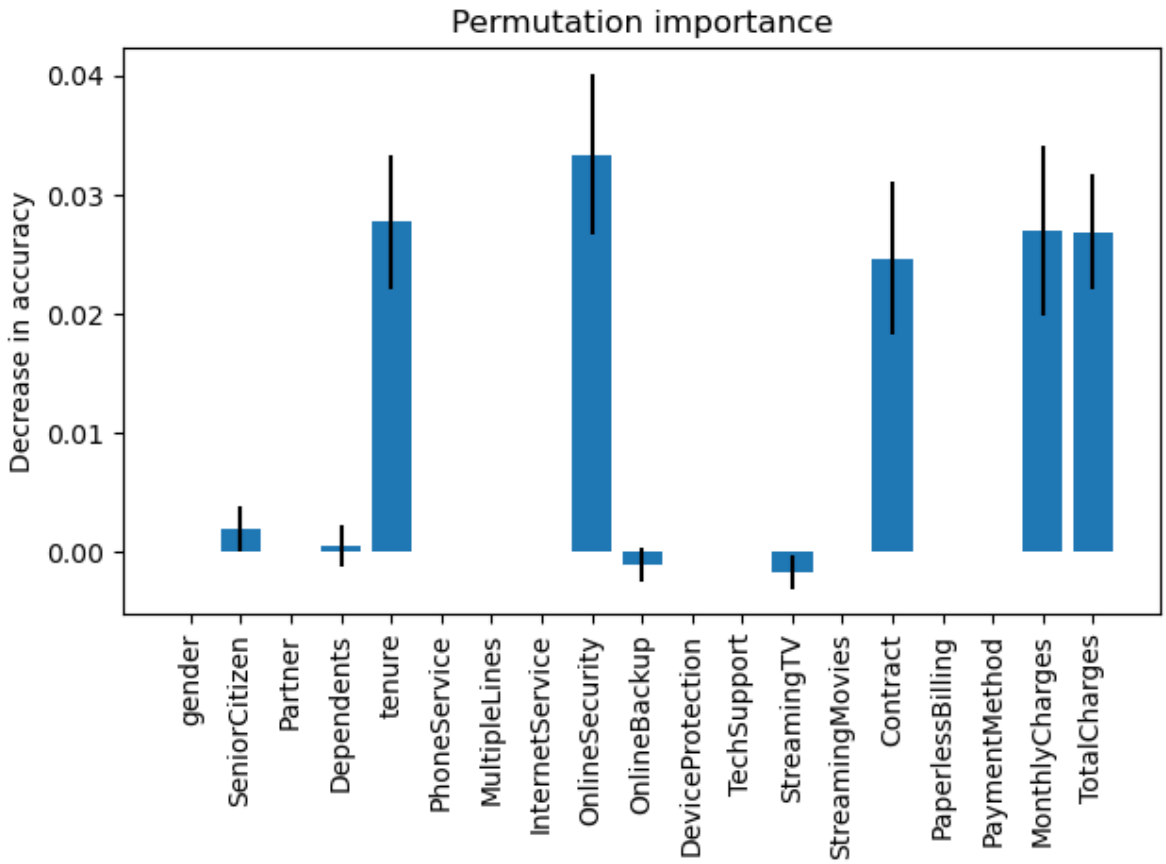


Figure 2: Permutation importance for each feature (test accuracy, 30 repetitions)

Five features stand out clearly: `OnlineSecurity` (0.0334), `tenure` (0.0277), `MonthlyCharges` (0.0270), `TotalCharges` (0.0269) and `Contract` (0.0247). When one of these is shuffled, accuracy drops by 2.5 to 3.3 percentage points. The error bars overlap between these five, so the ranking among them is uncertain. Ten of the features have an importance of 0.0000 (rounded to four decimals): `gender`, `Partner`, `PhoneService`, `MultipleLines`, `InternetService`, `DeviceProtection`, `TechSupport`, `StreamingMovies`, `PaperlessBilling` and `PaymentMethod`. For these, shuffling does not noticeably change the model's predictions, which is expected if the feature is not used in the tree. The remaining four have very low values: `SeniorCitizen` (0.0020), `Dependents` (0.0005), `OnlineBackup` (-0.0011) and `StreamingTV` (-0.0016).

4 Conclusion

We have implemented ID3 with entropy and Gini and maximum depth, and evaluated the implementation on the churn dataset with 7032 customers, split into a training set and a test set. With 5-fold stratified cross-validation on the training set, the best combination was `criterion="gini"` and `max_depth=5`, both for our implementation and for scikit-learn. Deep trees clearly overfit, while the choice between entropy and Gini matters little.

On the test set our implementation achieved an accuracy of 0.7825 and scikit-learn 0.7818. The F1 score for the churn class was lower, 0.5248 versus 0.5492, and shows that both models struggle to find the customers who actually leave the company. Permutation importance showed that `OnlineSecurity`, `tenure`, `MonthlyCharges`, `TotalCharges` and `Contract` are the features the model depends on the most, while ten features have no measurable importance for this model.

The results should be taken with a pinch of salt. The median split limits which features the tree can use, the model selection was done with accuracy on an imbalanced dataset. The scores therefore say something primarily about how this one model uses the features, and not about the real reasons why customers leave the company.